

Data Modeling – Building an NBA Player Network Graph

Day 3 Lab Follow-Along Notes – DataExpert.io Free Boot Camp

Video: Data Modeling - Building an NBA Player Network Graph - Day 3 Lab Instructor: DataExpert.io / Data With Zach

1. What Is the Goal of This Lab?

The goal of Day 3 is to introduce:

- Graph Data Modeling
- Relationship-based analytics
- Network structures in SQL
- Player-to-player connections
- Edge and node modeling
- Dimensional graph thinking

Instead of analyzing isolated rows, we now analyze:

relationships between entities

This is a major mindset shift.

2. Core Concept of Graph Modeling

Traditional relational modeling focuses on:

entities and attributes

Graph modeling focuses on:

entities and relationships

3. What Is a Graph?

A graph consists of:

Nodes

The entities.

Examples:

- players
 - users
 - customers
 - products
 - web pages
-

Edges

The relationships between nodes.

Examples:

- played together
 - follows
 - purchased with
 - friends with
 - linked to
-

4. NBA Graph Example

In this lab:

Nodes

NBA players.

Edges

Players who played together.

Example:

Kobe Bryant <----> Shaquille O'Neal

because they were teammates.

5. Why Graphs Matter

Graphs are powerful for:

- recommendation systems
 - fraud detection
 - social networks
 - influence analysis
 - dependency tracking
 - route optimization
 - knowledge graphs
 - relationship exploration
-

6. Traditional SQL vs Graph Thinking

Traditional SQL

Questions like:

How many points did a player score?

Graph SQL

Questions like:

Which players are most connected?
Who played with the most stars?
How far apart are two players?

Graph modeling unlocks relationship analytics.

7. Main Modeling Idea

The lab creates:

player-to-player relationship edges

using season/team level data.

8. Important Graph Terminology

Node

Entity in the graph.

Here:

player

Edge

Connection between two nodes.

Here:

two players who played together

Degree

Number of connections a node has.

Example:

LeBron played with many teammates

Higher degree = more connected.

Weight

Strength of connection.

Example:

number of seasons played together

9. Directed vs Undirected Graphs

Directed Graph

Relationship has direction.

Example:

Twitter follows
A -> B

Undirected Graph

Relationship is mutual.

Example:

teammates
A <-> B

The NBA player network is typically:

undirected

because if Kobe played with Shaq, then Shaq played with Kobe.

10. Main Dataset Logic

The lab uses player season data.

Example:

player	team	season
Kobe	Lakers	2000
Shaq	Lakers	2000
Fisher	Lakers	2000

From this we derive teammate relationships.

11. How Relationships Are Built

The key operation is:

```
self join
```

We join the table to itself on:

- same team
 - same season
-

12. Core Self Join Logic

Conceptually:

```
SELECT
  p1.player_name,
  p2.player_name
FROM players p1
JOIN players p2
  ON p1.team = p2.team
  AND p1.season = p2.season
```

This creates teammate pairs.

13. Why Self Joins Matter Here

Because relationships are formed between:

```
rows inside the same dataset
```

This is one of the most important SQL graph modeling patterns.

14. Preventing Duplicate Relationships

Without filtering:

```
Kobe -> Shaq  
Shaq -> Kobe
```

would both appear.

We usually prevent duplicates using:

```
WHERE p1.player_name < p2.player_name
```

This creates canonical ordering.

15. Why Canonical Ordering Is Important

Without it:

player_a	player_b
Kobe	Shaq
Shaq	Kobe

Both represent the SAME relationship.

Canonical ordering ensures:

one relationship = one row

16. Edge Table Structure

Typical edge table:

```
CREATE TABLE player_edges (  
  player_1 TEXT,  
  player_2 TEXT,  
  season INTEGER,  
  team TEXT  
);
```

17. Edge Granularity

Each edge usually represents:

one shared season together

Example:

p1	p2	season
Kobe	Shaq	2000
Kobe	Shaq	2001

18. Aggregated Relationship Strength

We can aggregate edges to calculate:

how strong the connection is

Example:


```
COUNT(*) AS seasons_together
```

19. Relationship Weight

Example:

p1	p2	seasons_together
Kobe	Shaq	8

This weighted edge is more meaningful than:

```
binary connected/not connected
```

20. Important SQL Pattern

Graph modeling in SQL often uses:

- self joins
- grouping
- edge generation
- aggregation
- adjacency style modeling

21. Why This Is Powerful

Once relationships exist, you can analyze:

- player communities
- career overlap
- teammate networks
- influence structures
- player centrality

22. Graph Thinking vs Table Thinking

Traditional tables answer:

What attributes does this entity have?

Graphs answer:

How is this entity connected?

23. Example Analytical Questions

Most Connected Player

Which player had the most unique teammates?

Strongest Teammate Pair

Which pair played together longest?

Team Connectivity

Which teams created the densest player network?

Career Bridges

Which players connected different NBA eras?

24. Real-World Graph Applications

Social Networks

node	edge
user	follows

Fraud Detection

node	edge
account	shared device

Recommendation Systems

node	edge
product	purchased together

Web Search

node	edge
webpage	hyperlinks

25. Key Insight

Graphs model:

interactions

not just:

properties

26. Relationship Explosion Problem

Self joins can create huge outputs.

Example:

A team with:

15 players

creates:

$15 \times 14 = 210$ relationships

before deduplication.

This is an important scalability concept.

27. Complexity of Graph Generation

Relationship generation often grows:

quadratically

This becomes expensive at large scale.

28. Optimization Concepts

To optimize graph creation:

- filter early
 - reduce duplicates
 - aggregate efficiently
 - partition by season/team
 - avoid unnecessary joins
-

29. Incremental Graph Building

Graphs can also be built incrementally.

Example:

```
today's new relationships only
```

instead of rebuilding entire graph.

Very common in production pipelines.

30. Temporal Graphs

This lab also introduces:

```
time-aware relationships
```

because connections exist during seasons.

Example:

```
Kobe and Shaq connected from 1996-2004
```

This becomes a temporal graph.

31. Graph Warehouse Thinking

The lab combines:

- dimensional modeling
- historical tracking
- relationship analytics

This is advanced warehouse design thinking.

32. Important SQL Concepts Used

SELF JOIN

Core graph relationship generation.

GROUP BY

Used for edge aggregation.

COUNT()

Measures connection strength.

DISTINCT

Removes duplicate edges.

LEAST() and GREATEST()

Sometimes used for canonical ordering.

Example:

```
LEAST(player1, player2)  
GREATEST(player1, player2)
```

Ensures consistent edge representation.

33. Common Beginner Confusion

Beginners often think:

```
graphs require graph databases
```

Not true.

A lot of graph analytics can be modeled in:

- PostgreSQL
- BigQuery
- Snowflake
- Spark
- Redshift

using relational techniques.

34. Why This Lab Matters

This lab teaches:

relationships are first-class analytical objects

That is a very important data engineering mindset.

35. Connection to Previous Labs

Day 1

Focused on:

cumulative historical storage

Day 2

Focused on:

state transitions over time

Day 3

Focused on:

relationships between entities

Together they build foundational data modeling intuition.

36. Real Industry Relevance

Graph-style thinking appears everywhere:

- LinkedIn connections
 - Uber route systems
 - Amazon recommendations
 - fraud rings
 - cybersecurity attack graphs
 - identity resolution
 - dependency mapping
-

37. Most Important Mental Shift

Traditional analytics:

entity-centric

Graph analytics:

relationship-centric

This is the core conceptual transition.

38. Powerful Intuition

Think of graph modeling as:

mapping the structure of interactions

instead of merely storing rows.

39. Typical Graph Pipeline Steps

1. Identify nodes
2. Identify relationships
3. Generate edges
4. Remove duplicates
5. Aggregate edge strength
6. Analyze network properties

This lab essentially walks through that pipeline.

40. Final Big Takeaway

Day 3 teaches that:

Data becomes far more valuable when relationships are modeled explicitly.

Tables tell you:

what exists

Graphs tell you:

how everything is connected

That is the fundamental idea behind graph data modeling.